

1. What Python Is & Why It's Used in AI

Python makes it easy to quickly prototype algorithms!

Python was created by Guido van Rossum (1989) and designed to be:

simple and readable

powerful

open-source

quick to develop with

It was inspired by the ABC language and named after Monty Python.

Why Python is popular in AI

concise syntax → faster coding

garbage collection → memory handled automatically

portable → runs on all major OS

built-in structures: lists, dictionaries, sets

strong ecosystem:

NumPy (numerical computing)

Pandas (data analysis)

Scikit-learn (ML)

PyTorch / TensorFlow (deep learning)

These libraries make Python ideal for:

search algorithms

machine learning

data processing

simulations

2. Python Style & Philosophy

Key design philosophy:

readable code matters

consistency matters (PEP 8 style guide)

simple > complex

one clear way to do things

Example PEP 8 rules:

4-space indentation

variable names:

lower_with_underscores

< 80 character lines

3. Running Python

REPL (interactive mode)

Command:

python3

Used for:

testing code quickly

experimenting

debugging

Example:

```
>>> [2*i for i in range(10)]
```

```
[0,2,4,6,8,10,12,14,16,18]
```

Scripts

Run files:

python3 script.py

Homework for AI course will be submitted as scripts.

IDE:

Pycharm, VS Code, Eclipse. Great for creating code files, running them, and using their debugging tools

4. Python vs Java

Python:

whitespace defines blocks

dynamic typing: variables don't have a predefined data type on declaration but rather inferred by the interpreter when runned based on the value assigned to them. Variables can automatically change their data type at will when needed.

fewer lines of code

No unary operators like – or ++

Java:

curly braces

static typing

verbose

Example:

Java:

```
System.out.println("Hello, World!");
```

Python:

```
print("Hello World!")
```

5. Data Types & Objects

Everything in Python is an object.

Common types: int, float, str, list, tuple, dict, bool, None

Strong typing: throw Type Errors when two values are incompatible operated with each

```
"foo" + 2 # error
```

Dynamic typing:

```
x = "foo"
```

```
x = 2
```

6. Math & Operators

Arithmetic:

```
+ - * /
```

Power:

```
**
```

Modulo:

```
%
```

Comparisons:

< <= >= == (Checks whether two variables contain the same value as each other) !=, is (checks whether variables point to the same object or space in memory. Shallow or deep copies)

Logical:

and or not

7. Strings

Creation:

```
"hello"
```

```
'hello'
```

```
"""multi-line"""
```

Formatting:

```
"%s can speak %d languages" % ("C3PO", 6000000)
```

Immutable:

```
x = "foo"
```

```
x.strip() # creates new object
```

8. Mutability

Immutable: strings, tuples (can't modify the elements of the same strings or tuples once created. Must create a new String or Tuple reflecting the changes)

Mutable: lists, dictionaries

Example:

```
x = [1,2,3]
```

```
y = x
```

```
x.append(5)
```

```
# both change
```

9. Sequences

Lists

```
li = [1,2,3]
```

```
li.append(4)
```

```
li.sort()
```

Tuples

Immutable:

```
tu = (1,2,3)
```

Strings

Sequence of characters.

10. Dictionaries

Mapping:

```
d = {'user': 'RDR2', 'pswd': '1234'}
```

Operations:

```
d.keys()
```

```
d.values()
```

```
d.items()
```

Fast lookup:

average O(1)

Hashing scheme used for dictionaries

Useful in AI:

state representations

heuristic tables

11. Functions

Definition:

```
def my_function(x):
```

```
    return x+1
```

Multiple return values as a tuple of values:

```
return mean, median, mode
```

Default args:

```
def f(a,b=3)
```

Keyword args:

```
f(b=3,a=2)
```

Variable args:

```
*args
```

```
**kwargs
```

No function overloading

Functions are first-class:

can pass into functions

can return from functions

12. Higher Order Functions

Map:

```
map(int, ['1','2'])
```

Filter:

```
filter(is_even, [1,2,3])
```

Lambda:

```
lambda x: x%2==0
```

13. Classes & OOP

Example:

```
class Student:
```

```
    def __init__(self,name):
```

```
        self.name=name
```

Concepts:

inheritance

subclasses

overriding methods

14. Magic Methods

Special behavior:

`__init__`

`__str__`

`__len__`

`__add__`

Enable:

operator overloading

printing objects

Called differently than standard python functions

15. Duck Typing

If it behaves like something, it is treated like it. Polymorphism for Python

Example:

`entity.fly()`

Works for:

duck

airplane

Fails for:

whale (no fly method)

16. Loops

For loop:

`for x in range(5):`

17. List Comprehensions

Replace loops:

`[x*x for x in nums]`

With filter:

`[x for x in nums if x>4]`

Very common in AI assignments.

18. Iterators

Objects with:

`__iter__`

`__next__`

Used by:

`for item in iterable`

Memory efficient.

19. Generators

Use yield.

`def f(n):`

`yield n`

Benefits:

lazy computation

memory efficient

used in search algorithms

Important for AI:

graph traversal

streaming data

Generators pause the function when the 'yield' keyword is encountered with the associated value. Return the yield value to where the function is being called as part of an element of a larger iterable object. Flow of control returns back to the line below of the 'yield' statement in the generator function to resume the function again. Can easily convert

generators into iterables at will when called (list, tuples, sets).

20. Imports & Packages

Module: .py files that can be imported into your code.

Import module:

`import math`

`from math import sqrt`

("from the math.py file, grab the sqrt function")

Alias:

`import numpy as np`

Install:

`pip install package`

21. Example: Graph Search (AI relevance)

Slides include a directed graph class and generator search function.

Key idea:

traverse nodes

don't revisit

yield nodes

This directly connects to:

DFS

BFS

search homework

22. Profiling Code

Measure speed:

`time.time()`

`timeit`

`cProfile`

Important for: algorithm efficiency and AI performance tuning

Why AI Exists as a Field

Artificial intelligence is concerned with building systems that can perceive, reason, learn, and act effectively in complex environments.

Artificial Intelligence - A Mod...

The field spans:

reasoning

learning

perception

language

decision-making

robotics

AI is both:

a scientific effort to understand intelligence

an engineering effort to build intelligent agents

2. What Is Artificial Intelligence? There is no single definition of AI — historically it has been framed in four ways based on two dimensions:

Dimension 1: Human vs Rational

Human: imitate human thinking/behavior

Rational: do the optimal or correct action mathematically

Dimension 2: Thinking vs Acting

Thinking: internal reasoning processes

Acting: observable behavior

This yields four major AI paradigms:

3. Acting Humanly — The Turing Test

Alan Turing reframed:

"Can machines think?" → "Can machines imitate human conversation?"

A machine passes if a human evaluator cannot distinguish it from a human through conversation.

To pass, an AI must integrate:

Natural language processing

Knowledge representation

Automated reasoning

Machine learning

A Total Turing Test also requires:

computer vision

robotics

This covers most AI subfields.

4. Thinking Humanly — Cognitive Modeling

Goal:

simulate actual human thinking

Methods:

introspection

psychological experiments

brain imaging

Key idea:

Build programs that mirror human reasoning steps.

Example:

General Problem Solver (Newell & Simon)

Led to:

cognitive science

5. Philosophy Roots of AI

The slides trace AI questions back centuries.

5.1 Descartes — Dualism

Mind and body are separate.

Mind = non-physical thinking entity

Body = mechanical system

Key claim:

Machines could imitate behavior but lack true mind.

Descartes questioned:

how do we know others aren't automata? 01-AI-In-Philosophy.pptx

5.2 Early Automata

17th–18th century mechanical "humans."

Idea:

bodies as machines

behavior can be engineered

These were early inspirations for AI.

5.3 Descartes' Test for Intelligence

He believed machines fail because:

cannot truly use language

cannot reason across situations

This anticipates later debates on AI intelligence.

01-AI-In-Philosophy.pptx

6. Alan Turing — Operationalizing Intelligence

Instead of defining thinking, Turing proposed measuring it.

Key idea:

Intelligence judged by observable behavior.

Imitation Game:

human judge

human + machine conversation

if indistinguishable → machine intelligent

Important:

correctness not required

human-like response required

7. Criticism — John Searle's Chinese Room

Argument:

passing Turing Test ≠ understanding

Scenario:

person manipulates Chinese symbols using rules

produces correct responses

but does not understand Chinese

Conclusion:

symbol manipulation ≠ intelligence

This challenges:

strong AI claims

Strong vs Weak AI

Searle:

brains cause minds → computers cannot have minds

01-AI-In-Philosophy.pptx

8. Dennett — Brain in a Vat

Thought experiment:

brain separated from body

consciousness preserved

later simulated in computer

Question:

what makes biological processing different from computational processing?

Challenges:

identity

consciousness

embodiment

9. Early AI Programs — ELIZA

ELIZA simulated therapy conversations using pattern matching.

It:

appeared intelligent

lacked real understanding

Supports:

Chinese Room criticism

symbolic AI limits

01-AI-In-Philosophy.pptx

10. Limits of AI

AI faces philosophical and technical constraints:

Key concerns:

intelligence vs simulation

consciousness

understanding vs pattern recognition
embodiment requirements

AI may:

perform tasks intelligently

without possessing awareness or mind

11. Can Machines Really Think?

Central debate:

View 1: Functionalism

If system behaves intelligently → it is intelligent.

View 2: Biological naturalism

Only brains produce minds.

View 3: Computationalism

Mind = information processing.

View 4: Skepticism

Machines imitate intelligence but lack understanding.

These debates shape:

ethics

AI safety

AGI research

1. Intelligent Agents – Core Idea

An agent is anything that perceives its environment through sensors and acts upon that environment using actuators. Examples include humans, robots, software agents, and simple systems like thermostats.

Key terminology:

Percept: the signal an agent receives from its sensors

Percept sequence: entire history of percepts

Action: what the agent does in response

2. Agent Function and Agent Program

An agent is formally described by an agent function:

$f: P \rightarrow A$

This maps a sequence of percepts (P) to an action (A).

Agent = architecture + program

Architecture: Physical system (robot body, computer hardware).

Program: Software that implements the agent function.

Ex agent: Human

Sensors: eyes, ears, nose

Actuators: hands, legs, mouth

3. Rational Agents

Rational behavior means doing the right thing based on available information.

A rational agent:

chooses actions that maximize expected performance

uses percept history and built-in knowledge

perceives its environment and behaves rationally (doing the right thing)

Important: Rationality does NOT mean omniscience. Agents act under uncertainty and incomplete information.

Consequentialism: we evaluate an agent's behavior by its consequences

Rational Choice Theory: the notion that individuals have bounded rationality. They can only make decisions based on the information available to them.

4. Performance Measure

Defines what counts as success for an agent.

Examples:

vacuum agent: clean squares, minimize movement

taxi agent: safety, legality, speed, comfort, profit

Rationality is defined relative to: performance measure

environment

percepts

actions available

5. Task Environments and PEAS Framework

Task Environment: a problem specification for which the agent is a solution. The specific world an AI agent perceives and interacts with sensors (to get data), actuators (actions), and plus the task it's trying to complete.

To design a rational agent, we specify the task environment using PEAS:

P – Performance measure E –

Environment A – Actuators S – Sensors

Example: Automated Taxi

Performance:

safe, fast, legal, comfortable, profitable

Environment:

roads, pedestrians, traffic

Actuators:

steering, brakes, horn

Sensors:

cameras, LiDAR, GPS

6. The Nature of Environments

Key properties used to classify environments:

Observability:

Fully observable: if a task agent sensors can detect all of the aspects of the environment that are relevant to the agent's choice of actions.

Partially observable: agents only have access to only the information that they can perceive via their sensors only access to some information about their surroundings but not to their entire global picture

7. Fully vs Partially Observable

Fully observable: Chess board visible.

Partially observable: Self-driving car (limited sensing).

8. Different types of Tasks Environments

Deterministic environment: Once an agent takes a specific actions, then the environment will be completely knowable after it completes that action

Nondeterministic environment: an agent can pick an action, but the outcome is uncertain

Games of chance

Stochastic Environment Randomness affects outcomes. We can quantify how likely each of the possible outcomes is

Episodic environment: the agent needs to process self-contained episodes. The agent observes the environment and then performs a single action, each action is not dependent on any previous action. Independent with one another.

Sequential Environment: Current decision affects future states or actions

Static environment: Environment does not change while agent reasons.

Dynamic environment: While agent reasons, the environment changes continuously.

Discrete environment : limited number of clearly defined actions an agent can take

Continuous environment: Infinite ranges (robot movement).

Single agent: Only one decision-maker.

One agent in the entire environment

Multi-agent: Multiple agents interact (competition or cooperation) in the same environment.

Pacman

9. Structure of Agents

Four main agent types:

Simple Reflex: Condition-action rules.

Model-Based: Maintains internal state.

Goal-Based: Chooses actions toward goals.

Utility-Based: Chooses actions maximizing utility.

10. Goal of AI Agent Design

Given a PEAS task environment:

Design agent function maximizing performance.

Implement an agent program on architecture.

Module 2 info

1. From Reflex Agents → Problem-Solving Agents

Reflex agents act only on the current percept and ignore percept history.

Reflex agents uses the incoming sensor data (percept) and uses the current percept in order to decide what action to take

A problem-solving agent, in contrast: plans ahead

reasons about future states

uses search to find a sequence of actions that leads to a goal

The computational process used is called search, and the resulting sequence of actions is called a solution.

2. What Is Search?

Search is the process of:

exploring possible states

finding a path from an initial state to a goal state

Process of planning a sequence of actions to take

Key idea:

The agent considers sequences of actions before acting. The overall process of trying to find a solution which consists of a sequence of actions from the current state our agent finds itself in, into some desired goal state

This is necessary when:

the environment is complex

the agent must plan ahead

the correct action depends on future consequences

3. Impact of Task Environments on Solutions

If an environment is:

fully observable

deterministic

known

Then: → a solution is a fixed sequence of actions. Solution: a path from the current state of an agent to the desired endpoint or final state. There can be multiple solutions to a search problem, but we find the most cost-effective or optimal solution.

If the environment is:

partially observable

nondeterministic

Then: → the solution may need to branch based on future percepts.

This distinction determines whether: simple planning works

or conditional planning is required.

4. Problem Formulation

A search problem must be formally defined before solving.

A problem is defined by:

Set of states: set of possible configurations, given elements in the environment, that a given agent can be in

Initial state: starting situation or configuration

Actions: possible moves from each state. A set to define the number of actions to move from state to another

Transition model: $\text{Result}(s, a) \rightarrow$ new state. Tell us all possible actions present to take at a given state. Transition from state to another state given an action

Path cost: cost of moving between states

Goal test: determines if state is goal

This formal structure turns real-world tasks into solvable computational problems.

5. Example Search Problem: 8-Puzzle

Goal: Arrange tiles into a specified order.

Search formulation:

States: all puzzle configurations (9! possibilities)

Actions: move blank tile up/down/left/right

Performance measure: minimize number of moves

Solution: sequence of tile moves

This illustrates:

large state spaces

combinatorial explosion

importance of efficient search.

7. Vacuum World Example

States:

agent location

dirt presence

In a two-cell version:

8 possible states total

Actions:

suck

move left/right

Goal:

all cells clean

Action cost:

each action costs 1

This example shows:

state representation

action modeling

transition effects.

03-Search-Problems.pptx

8. Solutions and Optimal Solutions

A solution:

sequence of actions leading to a goal state.

An optimal solution:

solution with lowest path cost.

Path cost must be:

additive

non-negative

Examples:

total distance

number of moves

time taken. 03-Search-Problems.pptx

9. The Art of Problem Formulation

Design choices strongly affect search difficulty:

Decide:

which properties to represent

what actions are allowed

how to measure cost

Poor formulation: → huge state space → slow search

Good formulation: → manageable search → faster solution discovery.

10. State Space

State space:

all states reachable from the initial state.

Key concepts:

Path:

sequence of actions between states.

Frontier:

set of nodes available for expansion.
The set of states that we can transition to based upon applying each and every action to the current state.

Solution:

path from initial state to goal state.

State spaces grow exponentially when:
many actions per state
deep search trees.

11. Abstraction in Search

Real world is extremely complex.

Search requires abstraction:

Abstract state:

represents many real-world configurations.

Abstract action:

represents many physical actions.

Example: Arad → Zerind

represents multiple actual driving routes.

Abstraction is valid if:

the abstract solution corresponds to a real-world solution.

12. Tree Search Framework

General algorithm:

Initialize frontier with initial state

Loop:

if frontier empty → failure

choose node to expand
(strategy-dependent)

if node contains goal → return solution
expand node and add successors to frontier

Search strategy determines:

which node to expand next
performance of algorithm.

General problem in Tree Search:
repeated visiting of same states,
resulting in higher time complexity

Graph search: uses an auxiliary data structure to record the list of already visited states. If a potential child node state has already been visited, don't visit it again as it has already been visited.

Higher space complexity of graph search than tree search.

13. Nodes vs States

State:

representation of a configuration.

Node:

structure used in search tree.

Node contains:

state

parent node

children

action

path cost

depth

Important: States do NOT store parent or depth — nodes do.

14. Search Algorithms (Overview)

Search algorithms differ by:

node expansion order

memory usage

guarantees

Two major classes:

Uninformed search:

no domain knowledge

"Blind search algorithm"

Uses only information available in problem definition

Doesn't use any external knowledge about where the goal is located.

Informed search:

uses heuristics

This section sets foundation for BFS, DFS, UCS, A*.

15. Uninformed Search (Intro)

Strategies that use only:

initial state

successor function

goal test

path cost

No heuristics.

Examples:

breadth-first search: Exploring all nodes in a tree or graph level by level.

Implemented as a FIFO Queue. Does BFS guarantee returning the shortest path to a solution? Yes, if the cost of every action is equivalent. All paths could cost only 1.

depth-first search: Frontier is a LIFO QUEUE.

18. Depth-Limited Search (DLS)

Depth-Limited Search is a version of DFS with a maximum depth limit.

Instead of exploring indefinitely, the search:

stops expanding nodes after a fixed depth L

prevents infinite descent in infinite state spaces

Think of it as: DFS + safety boundary.

Why it exists

Regular DFS has a major problem:

can go infinitely deep

may never find a solution even if one exists at a shallow level

Depth-Limited Search avoids this by enforcing a cutoff.

Behavior

If solution depth $\leq$ limit: → DLS can find solution.

If solution depth $>$ limit: → DLS fails (cutoff occurs).

Completeness

Not complete unless:

limit $\geq$ depth of shallowest goal

Optimality

Not optimal.

Why?

deeper, cheaper paths may be ignored

DFS nature does not guarantee shortest path

Time Complexity

$O(b^l)$

b = branching factor l = depth limit

Space Complexity

$O(b \cdot l)$

Very memory efficient (like DFS).

Possible outcomes

DLS returns one of three:

solution

cutoff

failure

Cutoff means: → search stopped due to depth limit, not because no solution exists.

19. Iterative Deepening Search (IDS)

Iterative Deepening Search combines:

DFS space efficiency

BFS completeness

It repeatedly runs Depth-Limited Search with increasing limits.

Depth sequence: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ until goal found.

Idea

Instead of committing to one deep DFS path:

Search shallow levels first, then gradually explore deeper.

This mimics BFS ordering.

Why it works well

Most nodes in a search tree are near the bottom.

Repeatedly exploring shallow nodes:

adds small overhead

but gains completeness and optimality.

Completeness

Yes.

Guaranteed if:

branching factor finite

Optimality

Yes (for unit step costs).

Because:

shallowest goal found first (like BFS).

Time Complexity

$O(b^d)$

Same as BFS.

Even though nodes are re-expanded, the overhead is small.

Space Complexity

$O(b \cdot d)$

Same memory use as DFS.

This is the BIG advantage over BFS.

Why IDS is important

It gives:

BFS optimality

DFS memory usage

This makes IDS one of the most practical uninformed search strategies.

20. DLS vs IDS (High-yield comparison)

21. Common Exam Traps

Professors love these:

Trap 1 "Is DFS complete?" → No (in infinite depth spaces).

Trap 2 "Is DLS complete?" → Only if limit $\geq$ goal depth.

Trap 3 "Is IDS inefficient because it repeats work?" → No — repeated work is small vs benefits.

Trap 4 "Which algorithm is best when memory is limited but optimal solution required?" → Iterative Deepening Search.

22. When to Use Each

Use Depth-Limited when:

you know max solution depth

you want to prevent infinite DFS

Use Iterative Deepening when:

depth unknown

memory limited

want optimal shallow solution

This is why IDS is often preferred over BFS in practice.

Module 3

1. Why Informed Search Exists

Uninformed search algorithms (BFS, DFS, UCS):

use no knowledge about the goal

explore blindly

often expand huge numbers of nodes

Informed search introduces heuristics to guide exploration and gather knowledge about the remaining distance to a goal.

Core idea:

Use knowledge about the problem to prioritize promising paths.

Instead of exploring all nodes equally, informed search focuses on:

states likely closer to the goal.

This dramatically improves efficiency.

Search algorithms based on an evaluation function

Straight line distance: what's the straight line distance, ignoring any intermediate points between each of the other points head exactly in the direction of the destination without having to worry about any obstacles, cities, roads, etc. in your path. Usually an underestimate.

Does not guarantee to always find the optimal solution

Ex: greedy search is guaranteed to find solutions faster than uninformed search algorithms but is still not guaranteed to be optimal.

Uniform cost search

A tree search algorithm used for traversing or searching a weighted tree, tree structure, or graph. (Type of uniformed search algorithm)

Uses a Priority Queue

Expand node with lowest path costs

Frontier = priority queue ordered by $g(n)$

Problem with UCS: can expand in unnecessary opposite directions of the

goal state as it is still uninformed.

Path cost: how much does it cost to reach a paritruclar node from the intial node on our search tree

2. Heuristics: The Core Idea

A heuristic is:

a rule of thumb

an educated guess

an estimate that guides search

A heuristic function:

$h(n)$ = estimated cost from state n → goal.

Key properties:

not required to be exact

useful even when imperfect

speeds up search dramatically

Heuristic knowledge reduces:

search depth

number of explored nodes.

3. Best-First Search Framework

General approach:

Select the node with the lowest evaluation score:

$f(n)$

Frontier:

priority queue sorted by $f(n)$

Different algorithms depend on: → how $f(n)$ is defined.

This creates:

greedy search

A* search

uniform cost search (special case).

4. Evaluation Functions

Search is guided using:

$f(n)$ = estimated total cost of solution through n .

Key components:

$g(n)$ = cost from start → n $h(n)$ =

estimated cost from n → goal

Different search algorithms:

Uniform cost: $f(n) = g(n)$

Greedy best-first: $f(n) = h(n)$

A*: $f(n) = g(n) + h(n)$

This is a major exam concept.

5. Greedy Best-First Search

Greedy search expands: → node that appears closest to goal.

Uses: $f(n) = h(n)$

Focus:

ignores path cost so far

only cares about estimated distance to goal

Behavior

Fast and focused, but risky.

May:

get stuck

take long detours

produce non-optimal solutions

Completeness

No (can get trapped).

Optimality

No.

Strength

extremely fast in many problems.

Weakness

poor reliability

ignores real costs.

6. Example Heuristic

Straight-line distance:

common heuristic in map problems.

Used in:

Romania pathfinding example.

It estimates: distance to goal ignoring roads.

This provides: reasonable approximation.

7. A* Search — The Most Important Algorithm

A* combines:

real cost so far

estimated remaining cost

Formula:

$f(n) = g(n) + h(n)$

Interpretation:

$g(n)$:

how far we've come.

$h(n)$:

how far we estimate remains.

A* chooses nodes with: → lowest estimated total solution cost.

8. Why A* Works So Well

It balances:

exploration vs exploitation.

Greedy search:

focuses only on $h(n)$

Uniform cost:

focuses only on $g(n)$

A*:

considers both.

Result:

finds optimal solutions

with far fewer expansions than UCS.

9. Admissible Heuristics

A heuristic is admissible if:

$0 \leq h(n) \leq$ true cost to goal.

Meaning:

never overestimates.

Also called: optimistic heuristic.

Guarantee: A* with admissible heuristic → optimal solution.

This is one of the most tested exam theorems.

20. Why A* Is Considered the "Gold Standard"

A* is one of the most important search algorithms because it:

expands promising paths first

avoids exploring clearly expensive paths

guarantees optimal solutions when the heuristic is admissible

It is a best-first search algorithm where:

$f(n) = g(n) + h(n)$

This balances:

cost so far

estimated remaining cost

This is why it performs better than:

uniform-cost search (too cautious)

greedy search (too risky)

21. Frontier Behavior in A*

A* uses:

priority queue frontier

ordered by smallest $f(n)$

At each step: → expand node with lowest estimated total solution cost.

Important detail: Even if a goal appears in the frontier: → DO NOT immediately return it.

You must: → confirm no lower-cost path exists first.

22. Why Admissibility Matters (Intuition)

Admissible heuristic: $h(n) \leq$ true remaining cost.

Meaning:

optimistic

never overestimates

Why this guarantees optimality:

If a heuristic overestimates: → the optimal path may appear worse → gets delayed in expansion → algorithm may return suboptimal goal first.

If heuristic underestimates: → optimal path always stays competitive in frontier ordering.

Therefore: A* expands nodes in correct cost order.

23. Proof Sketch Idea for A* Optimality (Conceptual)

Key idea:

Let: A = optimal goal B = suboptimal goal

We want: A expanded before B.

Because:

$f(n)$ for ancestors of A $\leq$ true cost of A and:

$\text{true cost}(A) < \text{true cost}(B)$

Therefore:

$f(A) < f(B)$

Since A* always expands lowest $f(n)$: → ancestors of A expand first → A reached before B

Therefore: A* finds optimal solution first.

You don't need full proof on exam, but you must understand reasoning.

24. Heuristic Design via Relaxed Problems

Powerful exam concept.

To create admissible heuristic:

Remove constraints from original problem

Solve relaxed version

Use cost as heuristic

Why it works: Relaxed problem cost $\leq$ real problem cost.

Examples (8-puzzle):

Original constraints:

move only into adjacent blank

Relaxations:

Relaxation 1:

move tile to any adjacent square → Manhattan distance

Relaxation 2:

move tile anywhere → tiles out of place

Both:

admissible

underestimate real cost.

25. Dominance of Heuristics

Heuristic h_1 dominates h_2 if:

for every node: $h_1(n) \geq h_2(n)$

AND both admissible.

Dominating heuristic:

closer to true cost

expands fewer nodes

more efficient A*

Example: Manhattan distance

dominates tiles-out-of-place.

26. Best and Worst Admissible Heuristics

Best possible admissible heuristic: $h^*(n) =$ true cost to goal.

This leads to: → perfect search → minimal expansions.

Worst admissible heuristic: $h(n) = 0$.

This reduces A* to: → uniform-cost search.

27. Performance Impact of Heuristic Quality

Better heuristic → fewer node expansions → faster search.

Example behavior:

Poor heuristic: millions of expansions.

Good heuristic: hundreds.

Heuristic quality directly affects: time

memory feasibility.

28. A* vs Uniform Cost Search

Uniform cost: $f(n) = g(n)$

A*: $f(n) = g(n) + h(n)$

Uniform cost:

explores outward evenly

may move away from goal

A*:

biased toward goal

still considers cost so far

29. A* vs Greedy Search

Greedy: $f(n) = h(n)$

A*: $f(n) = g(n) + h(n)$

Greedy:

rushes toward goal

may take bad paths

A*:

hedges bets

balances path cost + estimate

30. Practical Uses of A*

You will likely see this on conceptual questions.

Used in:

GPS navigation

robotics path planning

video game AI movement

logistics and scheduling

airline resource planning

Reason:

optimal pathfinding

efficient computation

31. Contour Expansion Concept

Visualization idea:

Uniform cost:

expands circular cost contours.

A*:

stretches contours toward goal.

Greedy:

shoots straight toward goal.

This explains:

behavior differences

performance gains.

32. Major A* Exam Traps

Trap 1: Return goal immediately when seen. → wrong.

Trap 2: Admissible = exact. → false (can underestimate).

Trap 3: Greedy optimal. → false.

Trap 4: A* always faster than greedy. → false (greedy may expand fewer nodes but is risky).

Trap 5: Admissible heuristic must be consistent. → false.

10. Consistent (Monotone) Heuristics

Stronger condition:

$h(n) \leq \text{cost}(n \rightarrow n') + h(n')$

This is triangle inequality.

Also requires: $h(\text{goal}) = 0$

Implications:

heuristic decreases smoothly

A* graph search remains optimal

no need to revisit nodes

Consistency $\Rightarrow$ admissibility.

But: admissibility $\neq$ consistency.

11. Heuristic Types

Three cases:

Consistent + admissible → ideal

Admissible but inconsistent → optimal but inefficient

Inadmissible → not optimal

These distinctions often appear on exams.

12. Uniform Cost, Greedy, and A* Relationship

Key insight:

Uniform cost: $f(n) = g(n)$

Greedy: $f(n) = h(n)$

A*: $f(n) = g(n) + h(n)$

A* generalizes both.

If $h(n) = 0$: → A* becomes uniform-cost search.

If g(n) ignored: → becomes greedy search.

13. Shape of Search (Conceptual)

Slides illustrate:

BFS:

expands evenly outward.

UCS:

expands along cost contours.

A*:

expands toward goal intelligently.

Greedy:

rushes toward goal.

This helps visualize search behavior.

14. Designing Good Heuristics

Good heuristics:

approximate real cost closely

cheap to compute

never overestimate

Sources:

relaxed problems

abstractions

domain knowledge

Better heuristic: → fewer nodes expanded.

15. Tradeoff: Accuracy vs Cost

More accurate heuristic:

reduces search work.

But:

may be expensive to compute.

Balance:

fast heuristic

informative guidance.

16. Complexity of Informed Search

Worst case:

still exponential.

But: good heuristic drastically reduces:

branching factor

nodes explored

Heuristic quality directly impacts performance.

Module 4

1. From “Puzzles” to “Games”: Why Adversarial Search Is Different

So far in search (BFS/DFS/UCS/A*), you're usually solving a puzzle: the environment doesn't actively try to stop you.

Puzzle: Trying to find a solution, but not playing against a physical opponent

A game introduces an opponent whose objective conflicts with yours, so the “environment” becomes strategic (your outcome depends on your move and the opponent's response). The “simplest game environments” are often modeled as:

Multiagent, typically two-player, turn-taking, and zero-sum (one wins = other loses; draws possible).

Usually also: static, discrete, fully observable, and deterministic (for chess-like games).

Static: the agent can sit and think for as long as it needs them. The configuration of the game doesn't change while its thinking

Discrete: set of actions to take to transition from one state to another is finite

Fully observable: you see the complete configuration of the chessboard and you're not hiding things back from it

Strategic: Involves two or more agents who are against each other

Complicated: Like card games are usually not fully observable where you don't see the complete configuration of the game. Pacman

Why AI researchers like games (high-level motivation)

Games are clean “laboratories” for AI: State is easy to represent (board position).

Legal actions are crisp (rules).

But the search space is enormous (chess, Go). Your transcript's examples (Deep Blue, AlphaGo) are exactly the motivational story that leads into minimax + pruning.

2. Core Modeling: Game as a Search Problem (Game Trees)

Game tree: is the search tree that follows from applying every sequence of moves all the way to all possible terminal tickets

In adversarial search, we build a game tree:

Nodes = game states (board positions)

Edges = legal moves

Root = current state

Terminal states = game-over (win/lose/draw)

Utility (a number) is defined for terminal outcomes (classic: win=+1, draw=0, loss=-1 for MAX)

Key idea: Unlike regular search where you just want a goal, here you want the move that yields the best outcome assuming the opponent responds well.

Terminology you'll be tested on

MAX: “us” (the player we're choosing a move for); tries to maximize the utility.

MIN: opponent; tries to minimize MAX's utility.

Ply: one move by one player = one level in the game tree (MAX ply then MIN ply then MAX ply...).

Adversarial search: where one or more agents (opponents) have conflicting goals. These types of algorithms anticipate the counter-moves of opponents to find the optimal strategy for maximizing one's own gain while minimizing the opponents.

3. AIMA 5.1 — Game Theory (What you need for the midterm level)

AIMA's “game theory” framing is basically: in multi-agent settings, you

can think of agents as optimizing objectives that may be:

Fully opposed (zero-sum competitive)

Partly aligned (non-zero-sum → alliances, cooperation, negotiation)

Uncertain / mixed (randomness, hidden information, etc.)

For your midterm scope, the most important takeaway is:

The zero-sum, turn-taking, perfect-information model is the “starter case” where minimax applies cleanly.

When you move away from those assumptions (randomness / imperfect info / non-zero-sum), you need extensions (expectimax, etc.).

4. AIMA 5.2 — Optimal Decisions in Games: The Minimax Idea

The minimax “rule”

Don't play hope chess. Meaning: don't pick a move just because it looks good if the opponent cooperates. Assume the opponent responds optimally.

Minimax value definition (conceptual)

Don't take a move that looks the best for you without thinking ahead at what your opponent will do.

At a terminal node: return its utility.

At a MAX node: value is the max of children's values.

At a MIN node: value is the min of children's values.

If max wins, it receives a positive score. If min wins, it receives a negative score as min tries to reduce its score while max tries to increase its score. Draw: 0 score for both parties

Weighted sum: is useful in determining how good the current player is doing based on his current number of positions and value of his pieces.

So minimax is a recursive backup procedure:

Leaf utilities get propagated up the tree by alternating max/min.

What minimax returns

At the root (MAX's turn), minimax returns:

the best achievable utility given optimal opponent play and therefore the argmax move to choose.

Complexity (why we need pruning)

If branching factor is b and depth is m , full minimax is:

Time: $O(b^m)$ (exponential)

Space: depends on implementation; depth-first recursion is $O(bm)$ or $O(m)$ stack + storage choices.

This exponential blow-up is the reason alpha-beta exists.

5. AIMA 5.2/5.3 — Alpha-Beta Pruning: Same Answer, Fewer Nodes

Alpha-beta pruning is minimax with a trick: it avoids exploring branches that

cannot possibly influence the final decision.

The meanings of α and β (must-know)

During minimax, keep two bounds:

α (alpha): MAX's current lower bound on what it can guarantee (MAX can get at least α).

β (beta): MIN's current upper bound on MAX's outcome (MIN will allow MAX to get at most β).

Pruning conditions (must-know)

You stop exploring a branch when:

At a MAX node: if you find a value $\alpha \geq \beta$, MIN will avoid this path $\rightarrow$ prune remaining siblings.

At a MIN node: if you find a value $\beta \leq \alpha$, MAX will avoid this path $\rightarrow$ prune remaining siblings.

Refer to Muhesh Huddars video on youtube on Alpha beta pruning

What alpha-beta guarantees

Same move as minimax, just faster (with pruning).

Best-case (with perfect move ordering): effectively reduces exponential base; the classic result is that it can search roughly twice as deep as minimax in the same time.

Move ordering matters

If you explore "good" moves first, you tighten α/β earlier $\rightarrow$ prune more.

6. AIMA 5.3 — Heuristic Alpha-Beta (Cutoffs + Evaluation Functions)

Real games are too big to search to terminal states (chess/go). So we do cutoff search:

Cutoff search recipe

Search to depth limit d (fixed ply depth, or time limit).

Treat leaf-at-cutoff as terminal and apply a heuristic evaluation function

$Eval(s)$.

Run minimax/alpha-beta using those eval values.

Evaluation function intuition

Instead of asking "is this a win/loss?", we estimate "how favorable is this position for MAX?"

Examples (typical ideas):

Material balance (piece values in chess)

Positional features (king safety, center control, mobility)

Weighted sum of features (your transcript mentions Deep Blue using tons of features).

Horizon effect (classic pitfall)

When you cut off search, the agent may "delay" a bad outcome beyond the cutoff depth so the evaluation doesn't see it yet (AIMA discusses this as the horizon effect).

Mitigations (conceptual):

Extend search in "tactical" unstable positions (quiescence-like ideas).

Better evaluation.

Probabilistic cutoffs (AIMA mentions probcut-style ideas in this neighborhood).

7. AIMA 5.5 + Slides/Transcript — Stochastic Games and Expectimax

Some games include randomness (dice rolls, card draws). Now "the opponent" isn't the only uncertainty source.

Why minimax doesn't fit perfectly

Minimax assumes the next step is chosen by:

MAX (trying to maximize)

MIN (trying to minimize)

But in stochastic games, there are chance outcomes not controlled by either player.

Expectimax / Expectiminimax core idea

Replace some MIN-like nodes with chance nodes whose value is an expected value (weighted average).

The expectimax principle from your slides:

Max nodes: behave as usual (choose best child).

Chance nodes: compute expected utility = $\sum_i p_i \cdot V(\text{child}_i)$.

05-Expectimax-and-Utilities.pptx

This matches your transcript: chance nodes reflect average-case outcomes, not worst-case.

Why alpha-beta pruning gets harder

Alpha-beta works cleanly when MIN is an adversary choosing the worst case.

With chance nodes, you're averaging—so bounds are weaker and classic pruning isn't as straightforward (your transcript explicitly calls this out).

High-yield midterm distinctions

Minimax: worst-case optimal vs optimal opponent

Expectimax: optimal in expectation vs random outcomes / stochastic environment

Utility values become more important because expectation requires numeric outcomes that reflect preferences.

Part B — AIMA Chapter 16 (16.1–16.3): Decision Theory + Utility

8. Big Picture: Decision-Theoretic Agents

A goal-based agent says: goal vs not-goal (binary). A decision-theoretic agent uses:

Probabilities (beliefs about uncertain world)

Utilities (what it wants, on a continuous scale)

Picks actions by maximizing expected utility.

9. AIMA 16.1 — Maximizing Expected Utility (MEU)

Key formula (must-know)

For an action a , expected utility is:

$EU(a) = \sum_s P(\text{RESULT}(a)=s) \cdot U(s)$

(AIMA labels this as Eq. 16.1 in the chapter.)

Then the rational rule is:

$a^* = \arg \max_a EU(a)$

Why this matters

Handles uncertainty directly (unlike pure logical entailment).

Lets you compare outcomes that aren't strictly "goal achieved."

10. AIMA 16.2 — The Basis of Utility Theory (Preferences $\rightarrow$ Utility Function)

This section is about why it makes sense to represent preferences with a real-valued function.

Preference foundations

Assume an agent has preferences over outcomes/lotteries (not just states). To model a "rational" preference ordering, you need consistency properties (axiom flavor).

Key takeaways you're likely to be tested on:

Utility function $U(s)$ assigns desirability to states.

Lotteries (probabilistic mixtures of outcomes) let us connect probability + utility.

Expected utility of a lottery (core idea)

Utility of a lottery is a weighted sum:

$U(p_1s_1 + \dots + p_ns_n) = \sum_i p_i U(s_i)$

This is the "expected utility" structure that makes MEU possible.

Why this is nontrivial

Without consistent preferences, you can get irrational cycles (paying repeatedly to swap choices in a loop). AIMA uses this as motivation for rational preference axioms.

11. AIMA 16.3 — Utility Functions (Assessment, Risk, and Human Biases)

This is the "practical + human" part: utilities aren't automatically known, and people don't always act like EU-maximizers.

16.3.1 Utility assessment + utility scales

To use decision theory in a real system, you often need preference elicitation:

Ask a person to choose between options

Infer the utility function consistent with their choices

Important property:

Utility has no absolute scale; only relative comparisons matter.

Many transformations (affine transformations) preserve preferences.

16.3.2 Utility of money and risk attitudes

People usually have concave utility for money (diminishing returns).

Concave utility $\Rightarrow$ risk-averse behavior

Linear utility $\Rightarrow$ risk-neutral

Convex utility $\Rightarrow$ risk-seeking

Insurance is a classic example:

A risk-averse individual prefers a sure smaller loss (premium) over a small probability of a catastrophic loss.

A large insurer is closer to risk-neutral over small gambles relative to its reserves.

16.3.3 "Picking the best of k options" optimism effect

If you choose among many options based on noisy estimates, you can get systematic over-optimism:

The max of noisy estimates tends to be biased upward.

So you pick something that "looked best," and then reality disappoints.

This is a super testable conceptual point: selection bias from maximizing over estimates.

16.3.4 Human judgment vs rationality (normative vs descriptive)

Decision theory is normative (how agents should behave).

Humans can be predictably irrational (descriptive findings).

Named phenomena AIMA highlights:

Allais paradox (choices violate expected utility axioms)

Ambiguity aversion (prefer known probabilities over unknown)

Framing effect (wording changes decisions)

These are high-yield because they're recognizable "label + definition + example" items.

12. How This Ties Back to Games (Ch. 5 $\leftrightarrow$ Ch. 16)

This is a very midterm-ish synthesis:

Minimax assumes an adversary chooses outcomes (worst-case).

Expectimax assumes chance chooses outcomes (average-case).

Expected utility (Ch. 16) is the general tool for evaluating uncertain outcomes.

In stochastic games, utilities must reflect preferences over outcomes—not just win/loss—because expectation requires meaningful numeric comparisons.

Module 5

What CSPs Are and Why They Matter
Constraint Satisfaction Problems (CSPs) shift the focus of search from finding a path to finding a valid assignment.

Traditional search:

Focus = sequence of actions

Goal = reach a state via a path

CSP search:

Focus = assigning values to variables

Goal = find a complete, consistent assignment

The goal itself matters more than how you got there.

CSPs are specialized for:

identification problems

configuration tasks

scheduling and allocation

reasoning under constraints

CSPs are going to give us a general purpose way of using knowledge about constraints to cut off search early and to push us towards finding the solution faster

Key idea: Represent problems declaratively using variables, domains, and constraints $\rightarrow$ solve with general-purpose algorithms.

Formal Definition of a CSP

A CSP consists of three components:

Variables: $X_1, X_2, \dots, X_n$

Domains: $D_1, D_2, \dots, D_n$ Each domain contains possible values that can be assigned to a variable.

Constraints: $C_1, C_2, \dots, C_m$ Each constraint restricts combinations of values variables can take.

State: Assignment of values to some or all variables.

Consistent assignment: No constraint violations.

Complete assignment: All variables assigned.

Solution: Assignment that is both complete and consistent.

Core Examples of CSPs

Sudoku:

variables = cells

domain = $\{1 \dots 9\}$

constraints:

all numbers in each row different

all numbers in each column different

all numbers in each 3×3 box different

Map coloring:

variables = regions

domain = $\{\text{red, green, blue}\}$

constraints = adjacent regions must differ

Scheduling:

tasks must occur in certain orders

limited time/resources

VLSI design:

circuit component placement must satisfy spatial constraints

Astronomy scheduling:

telescope pointing constraints

energy and time constraints

CSPs allow us to massively reduce the search space of pruning paths automatically that violate the given constraints. Thus the time complexity and efficiency is very good for CSPs

Types of CSP Variables and Constraints

Variable types:

Discrete variables:

finite domain (colors, digits)

infinite domain (integers, times)

Continuous variables:

solved via linear programming

Number of variables or infinity number of variables available in the CSP

Constraint types:

Unary constraints:

apply to one variable

example: $SA \neq \text{green}$

Binary constraints:

involve two variables

example: $WA \neq NT$

Higher-order constraints:

involve 3+ variables

example: cryptarithmic puzzles

Soft constraints:

preferences, not strict requirements

CSP as a Search Problem

CSP can be solved using standard search ideas. Like using a search tree

Initial state:

Variables in our domain having no values assigned to them

Successor:

assign a value to an unassigned variable

Goal test:

complete + consistent

Path cost:

irrelevant (all assignments equal cost)

Commutative: if the order of the application of any given set of actions doesn't affect the outcome.

CSPs are commutative because when we're assigning values to variables, we can reach the same partial assignment no matter the order

Important property: All solutions occur at depth n (number of variables).

Branching Factor and Commutativity

Naive CSP tree: Branching factor: $n \times d$

Total nodes: $n! \times d^n$

Key insight: CSPs are commutative:

Order of assignments does not matter.

Therefore: We assign one variable at a time $\rightarrow$ reduces branching dramatically.

Search size becomes: d^n instead of $n! \times d^n$

Massive efficiency gain.

Backtracking Search (Core CSP Algorithm)

Backtracking = depth-first search over assignments.

Algorithm idea:

Pick a variable

Assign a value

Check constraints

Continue recursively

If no legal value $\rightarrow$ backtrack

This is:

complete

systematic

baseline CSP solver

Backtracking occurs when: A variable has no legal values remaining.

Improving Backtracking with Heuristics
Heuristics dramatically improve efficiency.

A. Variable selection heuristics

Minimum Remaining Values (MRV):
Pick variable with fewest legal values.
“fail fast” principle

Most Constraining Variable: Break MRV ties by choosing variable affecting the most others.

B. Value selection heuristic

Least Constraining Value: Choose value that eliminates the fewest options for other variables.

Chose the variable that borders the most of the other variables if all variables share a tie in constraints

Forward Checking

Forward checking = inference step.

When assigning a value:

remove incompatible values from neighboring variables' domains

If any variable loses all values:

failure detected early

backtrack immediately

This prevents deep exploration of doomed branches.

Constraint Propagation

Forward checking is limited.

Constraint propagation:

repeatedly enforce constraints

remove inconsistent values across variables

Search alternates between:

assignment (search)

propagation (inference)

Local Consistency

Node consistency: Apply unary constraints. Remove invalid values from domain.

Arc consistency: Apply binary constraints.

Definition: Variable X_i is arc-consistent with X_j if: For every value in $\text{domain}(X_i)$, there exists a compatible value in $\text{domain}(X_j)$.

If not: Remove inconsistent values.

Arc Consistency Algorithm (AC-3)

AC-3 enforces arc consistency systematically.

Steps:

Initialize queue with all arcs

Pop an arc ($X_i \rightarrow X_j$)

Remove inconsistent values from X_i

If X_i changes:

add all neighboring arcs back to queue

Continue until:

queue empty (consistent), or

domain empty (failure)

Complexity: Worst case: $O(n^2 d^3)$

If constraint graph planar: $O(n d^3)$

CSP Graph Representation

CSP = constraint graph.

Nodes: variables

Edges: constraints

Directed arcs: used for arc consistency propagation.

Disconnected components: independent subproblems.

Search + Inference Framework

CSP solving combines:

Search: assign values

Inference: propagate constraints

Key goal: reduce domain sizes early.

This:

prunes search space

speeds solving

enables solving large problems

Module 6

1) From Search Agents →

Knowledge-Based Agents

Earlier agents:

Problem-solving agents search over atomic states

States contain little internal structure (e.g., “Planet = Kessel”)

Limitation:

Cannot reason about properties

Cannot answer queries like:

“Nearest habitable planet”

“Cold planet for perishable cargo”

Logical agents:

Represent knowledge about the world explicitly

Use reasoning + inference to draw conclusions about how the world works

Decide actions based on derived facts

Key idea:

Intelligence = representation + inference over knowledge

2) Knowledge-Based Agents

Knowledge Base (KB)

A knowledge base is:

A set of sentences in a formal representation language

Sentences encode facts and rules about the world

Two types of sentences:

Axioms

Background knowledge

Always true

Derived knowledge

Obtained via inference

Used given sentences to derive new sentences

Agent interacts with KB via:

TELL

Add information to KB

Example: “There is a breeze at (2,1)”

ASK

Query KB

Example: “Is there a pit ahead?”

Agent cycle:

Observe percept

Add to KB (TELL)

Reason (inference)

Ask KB for safe action

Act

Knowledge based agents are going to use logic in order to represent the world and then to make inferences about pieces of it that we haven't yet observed

3) Representation Language

Natural vs Formal Language

Natural language:

English, Spanish, Arabic

Ambiguous

Formal language:

Logic symbols

Precise syntax + semantics

Example mapping:

4) Wumpus World (Classic Knowledge-Based Agent Example)

Environment:

Grid world (4x4)

Agent starts at (1,1)

Objects:

Wumpus (danger)

Pits (danger)

Gold/Luke (goal)

Percepts:

Stench → near Wumpus

Breeze → near pit

Glitter/Gasp → goal nearby

Bump → wall

Scream → Wumpus killed

Properties:

Partially observable

Sequential

Deterministic

Static

Single agent

Key insight:

Agent must infer hidden state from percepts.

Example reasoning:

No breeze at (1,1) → no pit in adjacent squares → safe to move

5) Logic Foundations

Syntax vs Semantics

Syntax:

Rules for forming valid sentences

Semantics:

Meaning of sentences

Truth values

6) Models and Possible Worlds

Model:

Assignment of truth values to all propositions

Example: Sentence α : “No pit at (2,2)”

Models:

All worlds where that statement is true

A collection of sentences in the k.b.

Alpha belongs to a model if it is true where alpha is some sentence

Notation:

$M(\alpha)$ = set of models where α is true

Adding knowledge:

Reduces possible worlds

7) Logical Entailment

Definition: $\alpha \models \beta$ (α entails β)

Meaning:

Whenever α is true $\rightarrow \beta$ must also be true

Models of $\alpha \subseteq$ models of β

Interpretation:

β logically follows from α

Example: "Cat" entails "Animal"

8) Inference

Inference:

Deriving new sentences from KB

Goal:

Determine if $KB \models \alpha$

Methods:

Model checking

Logical proof

Resolution

Forward/backward chaining

9) Propositional Logic

Atomic sentences

Represent facts:

$W1,3$ = Wumpus at (1,3)

FacingEast

Logical connectives

10) Truth Tables

Negation:

$\neg \text{True} = \text{False}$

Conjunction:

True only if both true

Disjunction:

True if at least one true

Implication: False only when:

P true, Q false

Interpretation:

"If P then Q" is violated only when promise broken.

11) Logical Equivalence

Two sentences are equivalent if:

True in same models

Examples:

$\alpha \wedge \beta \equiv \beta \wedge \alpha$

$\neg \neg \alpha \equiv \alpha$

$\alpha \rightarrow \beta \equiv \neg \alpha \vee \beta$

DeMorgan:

$\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$

Used for:

Rewriting formulas

Proof transformations

12) Inference Rules

Modus Ponens

From:

$\alpha \rightarrow \beta$

α Infer:

β

AND Elimination

From:

$\alpha \wedge \beta$ Infer:

α

Contrapositive

$\alpha \rightarrow \beta \equiv \neg \beta \rightarrow \neg \alpha$

13) Soundness vs Completeness

Sound:

Only derives true conclusions

Complete:

Derives ALL true conclusions

Ideal inference:

Both sound and complete

14) Resolution (Most Important Exam Topic)

Single inference rule:

Powerful + complete

Used with:

Conjunctive Normal Form (CNF)

Clause

Disjunction of literals:

$(A \vee B \vee \neg C)$

Resolution Rule

From: $(A \vee B)$ $(\neg B \vee C)$

Infer: $(A \vee C)$

Cancel complementary literals.

15) Conjunctive Normal Form (CNF) (Watch video on youtube for help)

Every propositional sentence can be converted to:

AND of OR clauses

Steps:

Remove biconditionals

Remove implications

Move negations inward

Distribute OR over AND

16) Resolution Algorithm

Goal: Check if $KB \models \alpha$

Procedure:

Convert $KB \wedge \neg \alpha$ to CNF

Apply resolution repeatedly

If empty clause derived $\rightarrow$ contradiction

Therefore KB entails α

Empty clause = proof complete

17) Horn Clauses

Restricted logic form:

At most one positive literal

Example: $\neg A \vee \neg B \vee C$

Equivalent: $A \wedge B \rightarrow C$

Important:

Efficient inference

Closed under resolution

18) Forward Chaining

Data-driven reasoning

Process:

Start with known facts

Apply rules repeatedly

Add inferred facts

Continue until query reached

Runtime:

Linear in KB size

Used when:

Continuous percept updates

19) Backward Chaining

Goal-driven reasoning

Process:

Start with query

Find rules producing query

Try proving premises

Repeat recursively

Used when:

Specific question must be answered

20) Monotonicity

Property:

Adding knowledge never invalidates old conclusions

I can add new facts into the k.b. That are entailed by it. And that will mean other facts can be entailed by it.

If: $KB \models \alpha$

Then: $KB \cup \beta \models \alpha$

Important for:

Logical consistency

21) Knowledge + Search Integration

Inference as search problem:

22) Why Logical Agents Matter

Compared to search agents: